

Governed Multi-Agent GraphRAG for Banking Regulatory Intelligence

Phase 2 (AWS V1)

From GraphRAG Proof of Concept to a Production-Oriented,
Governed LangGraph-Based Multi-Agent Architecture on AWS

LangGraph | Microsoft GraphRAG | OpenAI GPT-4.1 | DeepSeek | AWS ECS

Jingru Chen • Phase 2 Complete — September 11, 2026

The portfolio evolves from evidence-grounded GraphRAG retrieval into a governed, deployable multi-agent AI runtime.

PHASE 1 — FOUNDATION

Microsoft GraphRAG PoC

- 23-document banking / regulatory corpus
- Entity, relationship & community indexing
- Basic / Local / Global Search
- Evidence validation & citation checks
- Runtime guardrails
- Public Streamlit demonstration

ENGINEERING GAP

What Phase 1 exposed

- Retrieval quality can be uneven
- Citation presence $\neq$ claim support
- Exact identifiers / aliases need resolution
- Provider behavior differs
- Long-running Global Search needs production controls
- Recovery and escalation must be explicit

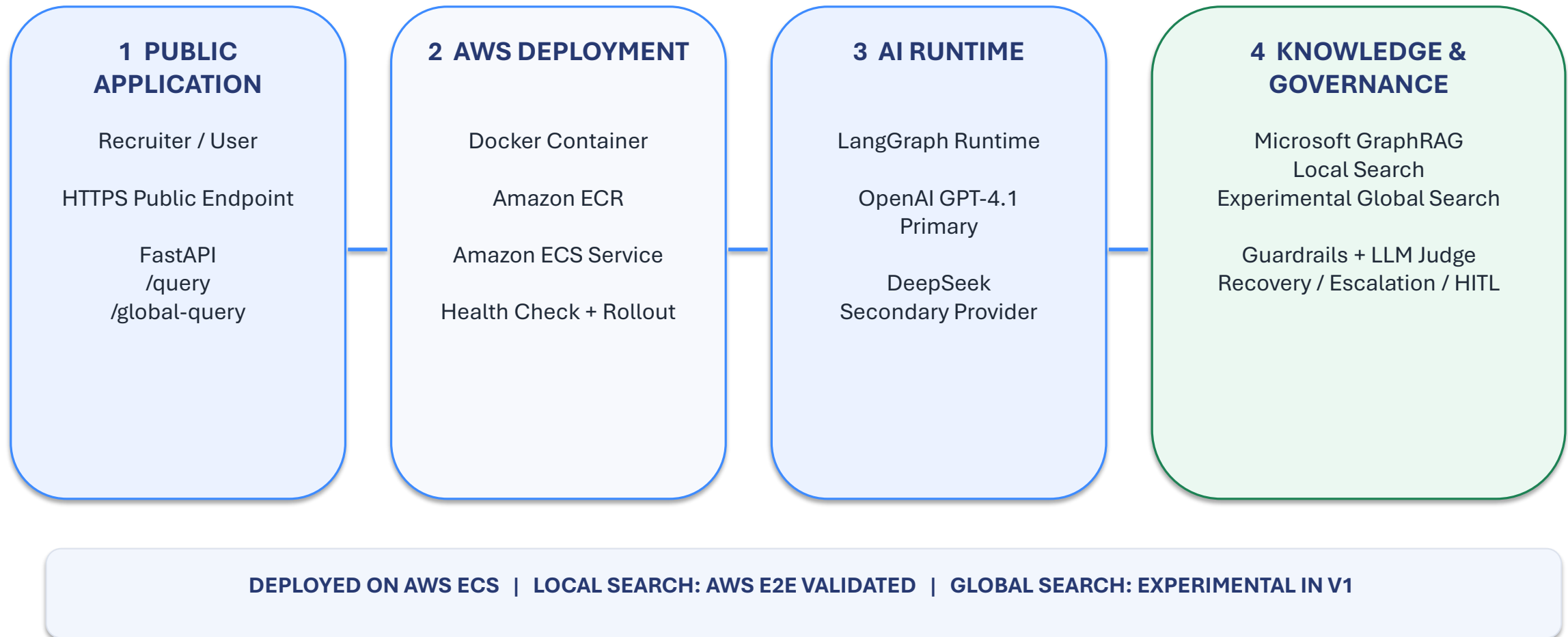
PHASE 2 (AWS V1) — DELIVERED

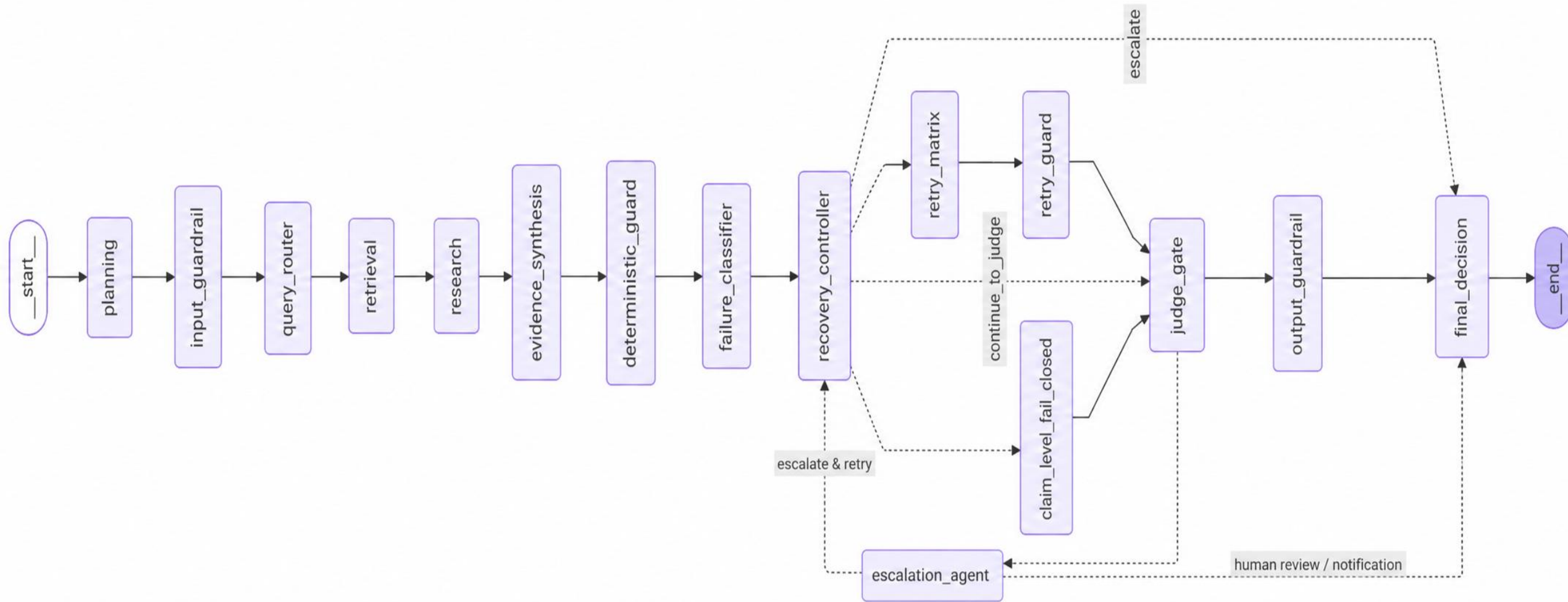
Governed Multi-Agent GraphRAG

- LangGraph orchestration
- Planning / research / validation workflow
- Deterministic guardrails + LLM judge
- Retry / recovery / escalation / human review
- OpenAI primary + DeepSeek secondary architecture
- Docker $\rightarrow$ ECR $\rightarrow$ ECS public AWS deployment

Engineering story: identify failure modes $\rightarrow$ design governed orchestration $\rightarrow$ validate end-to-end $\rightarrow$ deploy on AWS.

WHERE the system runs and WHAT production components are deployed; Slide 4 shows HOW LangGraph executes the governed workflow.





Defense-in-depth controls turn retrieval quality, provider behavior, and evidence sufficiency into observable runtime decisions.

1 PREVENT

Plan Guardrail

- Validate request / plan
- Bound unsupported behavior
- Preserve governed routing

2 DETECT

Deterministic Guard

- Evidence sufficiency
- Citation / provenance checks
- Missing-evidence detection
- Failure classification

3 RECOVER

Recovery Controller

- Continue to judge
- Retry matrix
- Retry guard
- Claim-level fail-closed

4 JUDGE

Independent LLM Judge

- Separate evaluation step
- Judge contract guardrail
- Approve or reject governed output

5 ESCALATE

Human Review

- PENDING_REVIEW is a valid governed outcome
- Escalate unresolved risk instead of hallucinating

Governance objective: make failure visible, recoverable, auditable, and safe—not merely maximize answer rate.

V1 was frozen only after container deployment, ECS rollout stability, health checks, and governed Local Search validation.

DEPLOYMENT PATH

Application + GraphRAG runtime



Docker container



Amazon ECR



Amazon ECS public service

AWS V1 FREEZE EVIDENCE

ECS rollout: COMPLETED

Desired: 1 Running: 1

Failed: 0 Pending: 0

Health endpoint: healthy

Orchestrator: LangGraph

Migration status: CLOSED_PASS

LOCAL SEARCH E2E

AWS Local Search: PASS

Example validated case:

Citigroup 2025 Q3 provision = \$2,259M

Guardrail: PASS

Judge: PASS

Final decision: APPROVED

GLOBAL SEARCH — IMPORTANT PRODUCTION FINDING

GraphRAG Global Search was validated in the application runtime (~9.4 min), but the public AWS synchronous path hit a 60-second ingress timeout (HTTP 504). V1 therefore exposes Global Search as EXPERIMENTAL rather than claiming synchronous AWS E2E success.

V2 architecture: start job → return job_id → background execution → status polling / result retrieval.

LOCAL: AWS E2E PASS

GLOBAL BACKEND: VALIDATED

AWS SYNC: 60s 504

V1: EXPERIMENTAL

Multi-Agent Observability & Traceability — LangSmith

LangSmith

Application

All applications

Search

Home

Tracing29

Engine

Monitoring1

Datasets & Experiments3

Evaluators

Annotation Queues2

Custom Apps

Prompts

Playground

Studio

Context Hub

Deployments

Sandboxes

LLM GatewayBeta

Settings

Personal

chen.jingru@gmail.com

Personal / Tracing / GraphRAG-Phase3-AWS-V1.3

GraphRAG-Phase3-AWS-V1.3

Tracing

Evaluators

Automations

Insights

Engine

New

Retention14d

Dashboard

+ New

Default View

Last 1 day

Threads

Traces

Runs

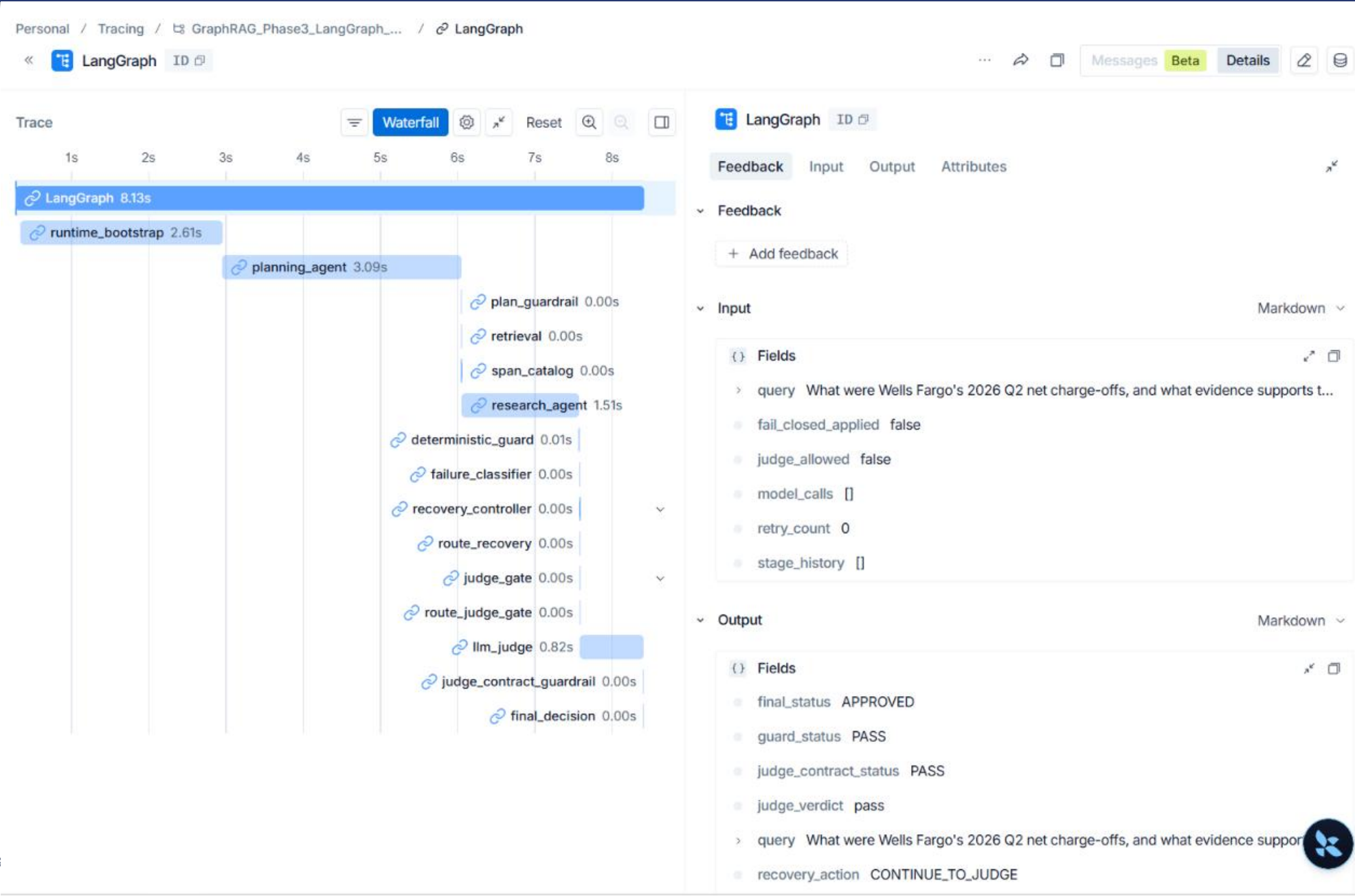
Input

Search input content

Add filter

</>

☐	☒	Name	Input	Output	Error	Start Time	Latency	Dataset	Ani
☐	☒	Global-Search-Experimental	Summarize the major reg...	# Major Regulatory and M...		9/11/2026, 7:36:57 PM	569.36s		
☐	☒	LangGraph	deepseek	REGULATORY		9/11/2026, 7:35:40 PM	9.16s		
☐	☒	LangGraph	openai	REGULATORY		9/11/2026, 7:35:29 PM	7.08s		
☐	☒	LangGraph	openai	REGULATORY		9/11/2026, 7:35:09 PM	8.71s		
☐	☒	LangGraph	openai	REGULATORY		9/11/2026, 7:34:50 PM	6.83s		
☐	☒	LangGraph	openai	REGULATORY		9/11/2026, 7:33:03 PM	8.06s		
☐	☒	LangGraph	openai	REGULATORY		9/11/2026, 7:32:39 PM	16.47s		
☐	☒	LangGraph	deepseek	DESCRIPTIVE		9/11/2026, 6:26:52 PM	8.58s		
☐	☒	LangGraph	deepseek	DESCRIPTIVE		9/11/2026, 6:25:06 PM	6.23s		
☐	☒	LangGraph	openai	DESCRIPTIVE		9/11/2026, 6:05:50 PM	11.28s		
☐	☒	LangGraph	openai	DESCRIPTIVE		9/11/2026, 6:04:33 PM	7.30s		
☐	☒	LangGraph	openai	DESCRIPTIVE		9/11/2026, 6:03:12 PM	13.26s		
☐	☒	LangGraph	openai	REGULATORY		9/11/2026, 5:57:49 PM	14.05s		



AWS V1 demonstrates end-to-end delivery; V2 focuses on production hardening, harness engineering, observability, and quantitative reliability.

AWS V1 — DELIVERED

- ✓ Governed LangGraph multi-agent runtime
- ✓ Microsoft GraphRAG retrieval integration
- ✓ OpenAI GPT-4.1 primary + DeepSeek secondary-provider architecture
- ✓ Deterministic guardrails + independent LLM judge
- ✓ Retry / recovery / escalation / human-review paths
- ✓ FastAPI + Docker + ECR + ECS public deployment
- ✓ AWS Local Search end-to-end validation
- ✓ Global Search production constraint identified and bounded

V2 — NEXT PRODUCTION HARDENING

1. Async Global Search: job orchestration + polling
2. True runtime OpenAI / DeepSeek routing
3. Agent Harness Engineering: contracts, structured I/O, context/evidence packs
4. Retry / timeout / fallback / escalation policies
5. LangSmith traces + operational metrics
6. Databricks telemetry / evaluation analytics
7. 20–30Q reliability benchmark + regression testing
8. AWS production hardening

Portfolio evolution: GraphRAG foundation → governed multi-agent runtime → AWS deployment → production reliability engineering.